RAMDEOBABA UNIVERSITY

School of Computer Science and Engineering

Shri Ramdeobaba College of Engineering and Management, Nagpur

An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur

[UNIVERSITY LOGO]

PROJECT REPORT

B. Tech. (Honor) in Computer Science and Engineering (Data Science)

~~/ B. Tech. (Honor) in Information Technology~~

~~Project~~ ~~Major Project~~

ON

CLOUD-BASED FOOD-DELIVERY ANALYTICS USING DATA ENGINEERING AND LARGE LANGUAGE MODELS

SUBMITTED BY

Amit Saw (Roll No. _____)

Ankit Kolhe (Roll No. _____)

Aryan Paradkar (Roll No. _____)

OF VII SEMESTER

UNDER THE GUIDANCE OF

Prof. / Dr. [Guide Name]

Project Guide, Department of Computer Science and Engineering

ACADEMIC YEAR: 2026–2027

CERTIFICATE

SHRI RAMDEOBABA COLLEGE OF ENGINEERING AND MANAGEMENT, NAGPUR

(An Autonomous Institute affiliated to Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project Report on "Cloud-Based Food-Delivery Analytics Using Data Engineering and Large Language Models" is a bonafide work of Amit Saw, Ankit Kolhe and Aryan Paradkar, submitted to the Rashtrasant Tukadoji Maharaj Nagpur University, Nagpur in partial fulfilment of the award of the Degree of Bachelor of Technology (B. Tech.) (Honor) in Computer Science and Engineering (Data Science) / Information Technology. It has been carried out at the Department of Computer Science and Engineering, Shri Ramdeobaba College of Engineering and Management, Nagpur during the academic year 2026–2027.

Date: _____ Place: Nagpur

Name of Guide Project Guide	External Examiner
Dr. A. M. Karandikar Associate Head Department of Computer Science & Engineering	Dr. M. B. Chandak Principal

STUDENT DECLARATION

We hereby declare that the project report entitled "Cloud-Based Food-Delivery Analytics Using Data Engineering and Large Language Models" is submitted as part of the requirements for the B. Tech. (Honor) in Computer Science and Engineering (Data Science) / Information Technology programme at Ramdeobaba University.

The work presented in this report is prepared by the undersigned students under the guidance of the project supervisor. All external sources, datasets, software tools, figures and technical material used in the project have been appropriately acknowledged. Where measured values were not available at the time of submission, the report contains clearly marked placeholders; these indicate measurements that are to be recorded from actual runs, and no unmeasured results are claimed as final.

Amit Saw _____ (Roll No. _____)

Ankit Kolhe _____ (Roll No. _____)

Aryan Paradkar _____ (Roll No. _____)

APPROVAL SHEET

This report entitled "Cloud-Based Food-Delivery Analytics Using Data Engineering and Large Language Models" by Amit Saw, Ankit Kolhe and Aryan Paradkar is approved for the degree of Bachelor of Technology (B. Tech.) (Honor) in Computer Science and Engineering (Data Science) / Information Technology.

_____	_____
Name of Guide Project Guide	External Examiner

_____

Dr. A. M. Karandikar
Associate Head, CSE

Date: _____ Place: Nagpur

ACKNOWLEDGEMENT

We express our sincere gratitude to Ramdeobaba University and the School of Computer Science and Engineering for providing the academic environment and facilities required to complete this project.

We are grateful to our project guide, Prof. / Dr. [Guide Name], for continuous guidance, technical suggestions and constructive feedback throughout the project. We also thank the Project In-Charge, Associate Head and Dean, School of Computer Science and Engineering, for their support.

We acknowledge the support of our classmates, laboratory staff, friends and family members who contributed directly or indirectly to the completion of this work. We also thank the open-source and documentation communities of Amazon Web Services, Snowflake, dbt Labs, Apache Software Foundation, Streamlit and xAI, whose resources made the implementation of this prototype possible.

ABSTRACT

Food-delivery platforms produce two very different kinds of data. Structured records such as orders, order items, restaurants, customers and menu items describe what happened in the business, while free-form customer reviews describe how customers experienced the food, service, pricing and delivery. These data sources provide complementary insight, but they are frequently processed in separate workflows, which makes it difficult to connect operational performance with the reasons customers report in their feedback.

This report presents the design and implementation of a cloud-based analytics prototype that combines a layered data pipeline with large language model (LLM) assisted access to business information. The pipeline uses Amazon S3 [20] for raw-file storage, Snowflake [21] as the analytical warehouse organised into Bronze, Silver and Gold layers, dbt for declarative SQL transformations and data-quality testing, and Apache Airflow for workflow orchestration. A Streamlit [22] interface supports two forms of interaction: review analysis, in which selected review text and a user instruction are sent to the Grok API, and natural-language-to-SQL querying, in which a question about the business is translated into a read-only query over curated analytical tables.

Because the review module uses direct prompting rather than a labelled classification pipeline or an embedding-based retrieval system, the report deliberately makes no claim of classification accuracy or retrieval quality. Evaluation is framed as pipeline checks, data-quality test outcomes, Text-to-SQL execution checks, qualitative inspection of review answers and operational observations. The contribution of the work is the integrated system design, the guardrails around natural-language database access, and a repeatable evaluation methodology into which measured results can be inserted.

Keywords: Cloud Data Engineering, Food-Delivery Analytics, Snowflake, dbt, Apache Airflow, Large Language Models, Natural-Language Querying, Streamlit

Contents

CERTIFICATE ii

STUDENT DECLARATION iii

ACKNOWLEDGEMENT v

ABSTRACT vi

LIST OF FIGURES x

LIST OF TABLES xi

LIST OF ABBREVIATIONS xii

1 INTRODUCTION 2

1.1 Background 2

1.2 Problem Statement 3

1.3 Objectives 3

1.4 Scope 4

1.5 Organisation of the Report 4

2 LITERATURE REVIEW AND RESEARCH GAP 6

2.1 Introduction 6

2.2 Review of Existing Approaches 6

2.2.1 Cloud Data Platforms and Lakehouse Architectures 6

2.2.2 Data-Engineering Pipeline Tooling 6

2.2.3 Language Models and Retrieval-Augmented Generation 7

2.2.4 Sentiment and Opinion Analysis of Reviews 7

2.2.5 Natural-Language Database Interfaces (Text-to-SQL) 7

2.3 Comparison of Existing Approaches 8
2.4 Limitations of Existing Work 8
2.5 Research Gap 8

3 PROPOSED SYSTEM AND METHODOLOGY 11

3.1 Proposed System 11
3.2 Data Acquisition 12
3.3 Data Preprocessing and Layered Organisation 12
3.4 Transformation and Data Quality with dbt 12
3.5 Workflow Orchestration with Airflow 13
3.6 Analytical Methods 13
3.7 Evaluation Metrics 13

4 SYSTEM DESIGN AND IMPLEMENTATION 16

4.1 System Architecture and Data Flow 16
4.2 Technology Stack 16
4.3 Database Design 16
4.4 Workflow Orchestration Design 18
4.5 Review Analysis Module 18
4.6 Natural-Language-to-SQL Module 18
4.7 User Interface 18
4.8 Implementation Workflow 20

5 RESULTS AND DISCUSSION 22

5.1 Experimental Setup 22
5.2 Functional Results 22
5.3 Pipeline and Data-Quality Results 22
5.4 AI Interaction Results 22
5.5 Discussion 24
5.6 Limitations 24

6 CONCLUSION AND FUTURE SCOPE **27**

6.1 Conclusion 27

6.2 Future Scope 27

REFERENCES **29**

APPENDIX A **31**

6.3 Environment Variables 31

6.4 dbt Data-Quality Tests Configured 31

6.5 Review-Analysis Prompt Template 31

6.6 SQL Guard Rules (Preliminary) 31

6.7 Recorded Versions 32

ANNEXURE A **33**

ANNEXURE B **34**

ANNEXURE C **35**

List of Figures

3.1 Overall system architecture of the proposed cloud-based food-delivery analytics platform 11

4.1 Implementation architecture and data flow realised by the Airflow DAG 16

4.2 Natural-language-to-SQL interaction in the Streamlit interface (screenshot to be inserted) 19

4.3 Dashboard layout of the Streamlit interface 19

List of Tables

2.1 Comparison of representative existing approaches 8

4.1 Software components used in the prototype 17

4.2 Gold-layer database schema (principal tables) 17

5.1 Functional test results (outcomes to be recorded from actual runs) . . . 23

5.2 Dataset and pipeline summary (values to be recorded from actual runs) . 23

5.3 AI feature test log (outcomes to be recorded from actual runs) 24

6.1 Annexure A: Publications and intellectual property 33

6.2 Annexure B: Student contribution statement 34

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, Durability
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
dbt	Data Build Tool
ELT	Extract, Load, Transform
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NL-to-SQL	Natural Language to Structured Query Language
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
S3	Amazon Simple Storage Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

Food-delivery systems operate as multi-sided marketplaces in which customers, restaurants and delivery partners interact through a digital platform. Every interaction leaves a data trail: orders and order items record transactions, restaurant and customer tables record entities and preferences, and reviews record free-form opinions about food quality, delivery time, pricing and service. Individually, each stream is useful; together they can explain not only what is happening in a business but also why customers perceive it that way.

This chapter introduces the domain, states the problem addressed by the project, lists the objectives, defines the scope of the prototype and describes the organisation of the remainder of the report.

1.1 Background

Modern analytical architectures increasingly separate storage from transformation and organise data into layered models to improve reliability and reuse. Lakehouse research highlights the value of combining flexible object storage with structured warehouse-style processing [1], and surveys of data-engineering tooling describe a mature ecosystem covering ingestion, transformation and orchestration [2]. At the same time, Transformer-based language models [3] and large language models [4] have made it practical to interact with text and data through natural language. Retrieval-augmented generation [5] and Text-to-SQL research such as the Spider dataset [6] have further expanded what non-specialist users can ask of a system.

These developments make it practical to design a single workflow in which raw delivery data is retained verbatim, transformed into tested analytical models, and then consumed by applications that mix dashboards, review summarisation and natural-language database access. This project focuses on that integration boundary between data engineering and AI-assisted analytics rather than on proposing a new language model. Data handling is performed in Python with the Pandas library [24], and review analysis is performed through the Grok API [23].

1.2 Problem Statement

In many prototype and educational analytics systems, operational records and customer reviews are handled in separate pipelines. This creates three practical limitations. First, it is difficult to link operational metrics such as delivery time or order value with the qualitative reasons customers report in reviews. Second, repeated loading, cleaning and analysis steps are performed manually, which is slow and error-prone. Third, business stakeholders who do not know SQL depend on technical staff for even routine questions.

The problem addressed in this project is therefore the design and implementation of a unified, layered cloud analytics prototype that ingests food-delivery data, curates it into tested analytical models, and exposes both structured metrics and review-derived insights through simple natural-language interfaces.

1.3 Objectives

The main objective is to build a single analytics prototype that connects cloud data processing with AI-assisted access to business information and customer feedback. The work is divided into the following objectives:

- To collect and store food-delivery data, including orders, customers, restaurants, menu items and customer reviews.
- To organise data into Bronze, Silver and Gold layers so that raw, cleaned and business-ready data remain separate.
- To use dbt to define transformations and run data-quality checks on analytical models.
- To use Apache Airflow to schedule and coordinate the data-pipeline tasks.
- To send selected review text to the Grok API so users can request summaries, common themes and review-based observations.
- To support natural-language questions over approved business tables by generating and executing read-only SQL queries.
- To present results through a Streamlit interface that is understandable to technical and non-technical users.

1.4 Scope

The project covers data acquisition, storage, layered transformation, data-quality testing, workflow orchestration, AI-assisted review analysis, natural-language querying and visualisation. The prototype is intended for academic and engineering demonstration on a provided dataset; it does not handle live orders, real payments or production customer accounts.

The review feature performs direct LLM-based analysis of supplied review text; it does not implement embedding-based retrieval and is not a validated sentiment classifier. Similarly, the Text-to-SQL feature is a guarded prototype, not a production database interface. Claims about accuracy, time savings or business improvement are made only where those outcomes have actually been measured and recorded.

1.5 Organisation of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 presents the literature review and research gap. Chapter 3 describes the proposed system and methodology. Chapter 4 discusses system design and implementation. Chapter 5 presents results and discussion. Chapter 6 concludes the work and presents future scope. The references, an appendix of configurations and prompt templates, and three annexures follow the chapters.

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

CHAPTER 2

LITERATURE REVIEW AND RESEARCH GAP

The literature review establishes the technical context of the proposed system by examining cloud data architectures, data-engineering tooling, language models and retrieval methods, sentiment analysis of reviews, and Text-to-SQL research. The review is used to justify the methodology adopted in Chapter 3 and to identify the gap this project addresses.

2.1 Introduction

Research relevant to this project falls into five themes: (i) cloud data platforms and lakehouse architectures; (ii) data-engineering pipeline tooling; (iii) language models and retrieval-augmented generation; (iv) sentiment and opinion analysis of restaurant and delivery reviews; and (v) natural-language database interfaces. Each theme is reviewed below, followed by a comparison of representative approaches and a statement of the research gap.

2.2 Review of Existing Approaches

2.2.1 Cloud Data Platforms and Lakehouse Architectures

Lakehouse research examines ways to bring data-lake flexibility and warehouse-style management into a common analytical architecture [1, 7]. The Delta Lake work demonstrates that ACID table storage can be built over cloud object stores, which makes layered batch pipelines practical without a traditional warehouse-only design [8]. For this project, the relevant lesson is architectural: raw data and curated analytical data have different purposes and should be managed deliberately, as separate layers rather than as one overwritten table.

2.2.2 Data-Engineering Pipeline Tooling

A survey of pipeline tools for data engineering [2] describes the distinct roles of ingestion, transformation and orchestration in reliable workflows, and production machine-

learning experience shows that data management, not model code, dominates pipeline effort [9]. Frameworks such as dbt [19] make transformations explicit, testable SQL models, while Apache Airflow [18] sequences and schedules them as directed acyclic graphs. These ideas inform the decision to treat S3, Snowflake, dbt and Airflow as distinct components rather than collapsing the pipeline into one script.

2.2.3 Language Models and Retrieval-Augmented Generation

The Transformer architecture [3] underpins modern large language models, whose few-shot capabilities [4] allow a user instruction and a block of supplied text to produce useful analysis without task-specific training. Retrieval-augmented generation (RAG) describes methods that provide language models with external evidence at response time [5], and surveys trace its evolution and current landscape [10]. General LLM surveys provide broader context on capabilities and limitations [13]. The current review module differs from a retrieval-augmented design: review text is supplied directly to the Grok API prompt and the answer is generated from that provided context, so this report describes it as direct LLM-based review analysis, not as a vector-search RAG implementation.

2.2.4 Sentiment and Opinion Analysis of Reviews

Work on sentiment analysis of restaurant reviews shows that Transformer models can be applied to review text, including in resource-constrained edge settings [11], and bidirectional pre-training remains a standard baseline [12]. Such studies typically require labelled training or evaluation data. Because this project has no labelled review dataset, conventional sentiment-classification accuracy is not claimed; the review module is evaluated by qualitative inspection of its answers instead.

2.2.5 Natural-Language Database Interfaces (Text-to-SQL)

Text-to-SQL research has produced large labelled benchmarks such as Spider [6], large-scale database-grounded evaluation such as BIRD [14], and schema-aware synthesis methods such as CHESS [15]. Earlier work includes reinforcement-learning query generation [16] and table pre-training [17]. These studies show both the promise and the difficulty of translating questions into valid SQL over real databases, and they motivate providing the model with a limited schema description and restricting the generated query path. They do not establish that a student prototype reaches benchmark accuracy; such a claim would require a defined test set and measured results.

2.3 Comparison of Existing Approaches

Table 2.1 compares representative existing approaches along their data source, main technique and typical limitation.

Table 2.1: Comparison of representative existing approaches

Approach	Data Source	Main Technique	Typical Limitation
Lakehouse platforms [1, 7, 8]	Cloud object stores	Unified batch and table storage	No built-in natural-language access
Pipeline tooling surveys [2, 9]	Mixed sources	Ingestion, transformation, orchestration	Tool-centric; AI access left to separate systems
Sentiment analysis of reviews [11, 12]	Labelled review corpora	Trained classifier	Requires labelled training data
RAG systems [5, 10]	External document stores	Retrieval plus generation	Requires retrieval infrastructure and indexing
Text-to-SQL research [6, 14, 15]	Schemas and questions	Schema linking and SQL synthesis	Benchmark accuracy does not transfer automatically
Dashboard / BI tools	Stored measurements	Visualisation	Business users still need SQL or report authors

2.4 Limitations of Existing Work

The reviewed approaches can involve challenges related to heterogeneous data sources, inconsistent sampling intervals, data quality, communication reliability and integration between monitoring and analytics. Dashboard systems may present measurements without analytical summaries, while LLM-based tools may present fluent answers without a reliable data foundation. Text-to-SQL systems can generate syntactically valid queries that still answer the wrong question, and keyword-level safety checks alone are not a sufficient security boundary. Sending review text to an external API additionally raises data-governance, cost and rate-limit concerns that purely internal pipelines do not face.

2.5 Research Gap

The research gap addressed by this project is the integration of layered cloud data processing, declarative transformation with automated data-quality tests, workflow orchestration, direct LLM review analysis and guarded natural-language database access in one lightweight, campus-demonstrable system. Existing work generally solves one or two of these problems well, but a single reference implementation that keeps raw, cleaned and business-ready data separate while offering both structured and unstructured natural-language interaction is not commonly available for the food-delivery

domain. This project contributes such an integrated prototype together with an honest evaluation methodology.

CHAPTER 3

PROPOSED SYSTEM AND METHODOLOGY

CHAPTER 3

PROPOSED SYSTEM AND METHODOLOGY

The proposed system consists of five components: ingestion, storage, transformation and quality, AI services, and presentation. Raw files are collected and loaded into cloud object storage, curated into layered warehouse tables, tested declaratively, orchestrated by a scheduler, and finally exposed through a single web interface that supports dashboards, review analysis and natural-language querying.

3.1 Proposed System

Figure 3.1 shows the overall architecture. Source files for customers, restaurants, menu items, orders, order items and reviews are uploaded to Amazon S3 and loaded into Snowflake, where the Bronze layer keeps the raw representation. dbt models clean and standardise the data into Silver tables and build curated Gold dimensions, facts and business marts. Data-quality tests run as part of the transformation step. Apache Airflow coordinates the sequence and schedules repeat runs. From the Gold layer, the Streamlit application serves dashboard metrics and the natural-language-to-SQL feature, while the review-analysis feature sends selected review text to the Grok API.

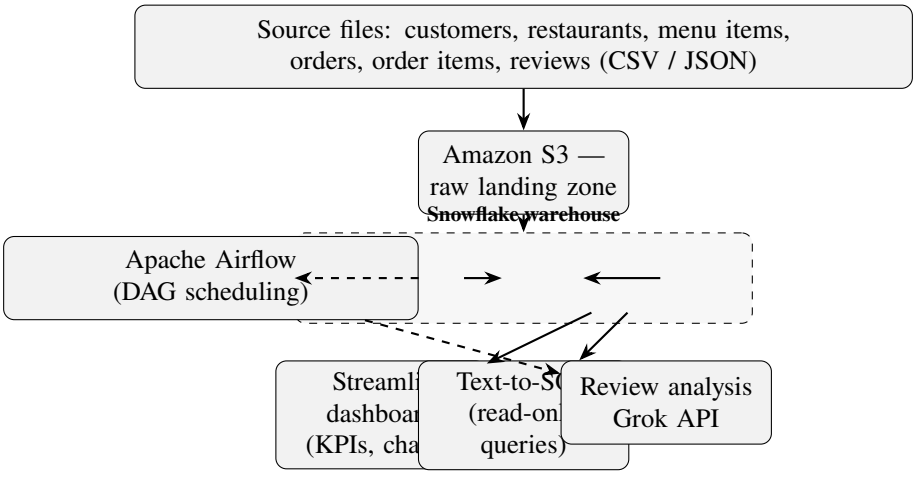


Figure 3.1: Overall system architecture of the proposed cloud-based food-delivery analytics platform

3.2 Data Acquisition

The dataset represents a food-delivery domain and includes customers, restaurants, menu items, orders, order items and reviews. Source data is prepared as files and stored in Amazon S3; Snowflake is used as the analytical warehouse. Each observation record is associated with a timestamp, an entity identifier and a measurement or text value. The exact dataset source, licence, row counts by table, date range and any synthetic-data generation steps are recorded in Chapter 5 and must be documented before submission.

3.3 Data Preprocessing and Layered Organisation

The pipeline applies a Bronze–Silver–Gold organisation. The Bronze layer retains source data with minimal changes so that it remains a reference point for checking what was originally loaded. The Silver layer applies cleaning and standardisation, including data-type corrections, consistent date and text formats, de-duplication and defined handling of invalid or missing values. The Gold layer contains curated dimensions, facts and business marts designed for analysis.

Where measurements are compared or scaled, min–max normalisation is applied as

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

where x is the original value and $x_{\min}$ and $x_{\max}$ are the minimum and maximum of the observed range. Preprocessing rules are fixed before final evaluation so that they do not introduce unintended bias into reported results.

3.4 Transformation and Data Quality with dbt

dbt organises warehouse transformations as versioned SQL models with declared dependencies, allowing staging and analytical tables to be built in a defined order. Data-quality tests check uniqueness of identifiers, non-null constraints, valid category values and relationships between foreign and primary keys. These checks help identify issues before data is used by downstream reports; the tests actually implemented and their outcomes are reported in Chapter 5.

3.5 Workflow Orchestration with Airflow

Apache Airflow coordinates the pipeline through a directed acyclic graph (DAG). A typical run loads data from the warehouse landing tables, executes dbt transformations, runs dbt tests and then allows downstream dashboard refreshes and AI services to proceed only when dependencies have succeeded. Airflow is responsible for scheduling and task coordination; it does not replace dbt's transformation logic, and a failed quality check must not silently publish unreliable outputs.

3.6 Analytical Methods

Descriptive statistics provide the initial understanding of the curated data. For n observations, the arithmetic mean is

$$\bar{x} \;=\; \frac{1}{n}\sum_{i=1}^{n} x_i .$$

Two AI-assisted methods are then applied. For review analysis, selected review text and the user's instruction are sent to the Grok API, which is asked to ground its answer in the supplied reviews. For structured business questions, the application provides the model with a description of approved tables and columns, receives a SQL query in a predictable format, validates it with a keyword-level check, and executes it through a read-only Snowflake role. No supervised machine-learning model is trained in this project, so no training procedure or classification accuracy is reported.

3.7 Evaluation Metrics

Evaluation combines functional and qualitative criteria. Functional checks record whether each end-to-end task completes. For the query path, the SQL execution success rate is

$$\mathrm{SER} \;=\; \frac{N_{\mathrm{executed}}}{N_{\mathrm{total}}} \times 100\,\%,$$

where N_{executed} is the number of generated queries that execute successfully and N_{total} is the number of questions attempted. Review answers are inspected against a fixed question set and judged for relevance, support by the supplied text and absence of unsupported claims. Regression metrics such as RMSE are defined for future predictive extensions:

$$\mathrm{RMSE} \;=\; \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2},$$

but they are not reported in this version of the project because no predictive model is included.

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

CHAPTER 4

SYSTEM DESIGN AND IMPLEMENTATION

The implementation is divided into ingestion, storage, transformation, orchestration, AI-service and visualisation modules. The architecture supports a modular design so that additional data sources, tests or analytical methods can be incorporated without redesigning the complete system.

4.1 System Architecture and Data Flow

Figure 4.1 shows the task-level data flow realised by the Airflow DAG. The sequence moves from extraction and raw loading through staged transformation and testing, and only then to downstream consumption by dashboards and AI services.

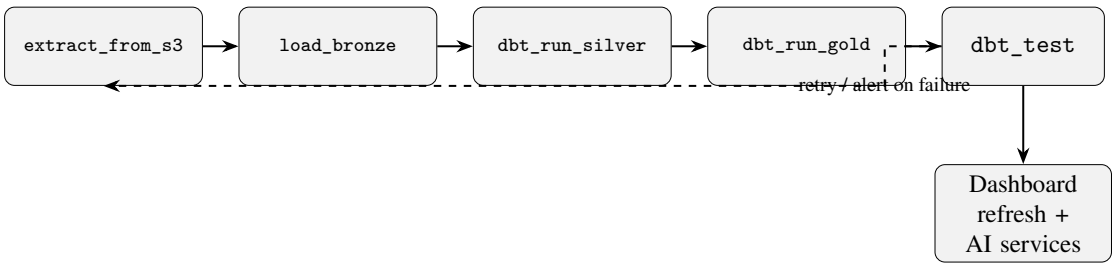


Figure 4.1: Implementation architecture and data flow realised by the Airflow DAG

4.2 Technology Stack

Table 4.1 lists the software components used in the implementation. Exact versions and configurations are recorded in Appendix A.

4.3 Database Design

The Gold layer uses a star-schema design. Table 4.2 summarises the principal tables and fields.

Table 4.1: Software components used in the prototype

Component	Purpose	Version
Python with Pandas	Data handling and preparation scripts	*[record version]*
Amazon S3	Raw-file landing zone	Managed service
Snowflake	Analytical warehouse and layered storage	*[record version]*
dbt Core	SQL transformations and data-quality tests	*[record version]*
Apache Airflow	DAG-based scheduling and coordination	*[record version]*
Snowflake Connector	Programmatic warehouse access	*[record version]*
Streamlit	Web interface for dashboards and AI features	*[record version]*
Grok API (xAI)	Review analysis and SQL generation	*[record model id]*

Table 4.2: Gold-layer database schema (principal tables)

Table	Column	Type	Description
`fact_orders`	`order_id`	VARCHAR	Order identifier (primary key)
	`customer_id`	VARCHAR	Customer foreign key
	`restaurant_id`	VARCHAR	Restaurant foreign key
	`order_ts`	TIMESTAMP	Order time
	`delivery_minutes`	NUMBER	Delivery duration
	`total_amount`	NUMBER	Order value
`dim_customers`	`customer_id`, `city`, `signup_date`	VARCHAR / DATE	Customer attributes
`dim_restaurants`	`restaurant_id`, `cuisine`, `city`, `rating`	VARCHAR / NUMBER	Restaurant attributes
`dim_menu_items`	`item_id`, `restaurant_id`, `price`	VARCHAR / NUMBER	Menu catalogue
`stg_reviews`	`review_id`, `order_id`, `rating`, `review_text`	VARCHAR / NUMBER	Cleaned review records

4.4 Workflow Orchestration Design

The Airflow DAG is configured with task-level retries and explicit dependencies, so that transformation tasks depend on successful loads, tests depend on transformations, and downstream refreshes depend on passing tests. Failures raise alerts instead of publishing outputs. Scheduling frequency is set for batch operation; near-real-time ingestion is left to future work.

4.5 Review Analysis Module

The review interface retrieves review records from Snowflake, selects a bounded sample of reviews per request, and sends the review text together with the user's instruction to the Grok API. The prompt requests concise summaries, recurring complaints, positive themes or grounded explanations, and instructs the model to use only the supplied text. The number of reviews per request, the truncation policy, the exact model identifier and the sampling method are documented in Appendix A. Responses are displayed in Streamlit as model-generated analysis; they are not treated as validated sentiment classifications, and personally identifying content is filtered before any text is sent to the external API.

4.6 Natural-Language-to-SQL Module

For structured business questions, the application supplies the model with a description of approved tables and columns and requests SQL in a predictable response format. The generated query is checked against a keyword-level rule set (for example, rejecting statements that do not begin with `SELECT` and rejecting DDL or DML keywords), then executed through a read-only Snowflake role restricted to approved views, with query timeouts and row limits. Results are displayed as a table and, where appropriate, as a simple chart. Figure 4.2 shows the intended interaction.

The keyword check is a preliminary safeguard, not a complete SQL parser or a substitute for database permissions; least-privilege roles remain the primary control.

4.7 User Interface

Streamlit provides the interface for the prototype. It brings together summary cards, time-series and category charts, result tables, the Text-to-SQL query form and the review-analysis interaction. Figure 4.3 shows the intended dashboard layout.

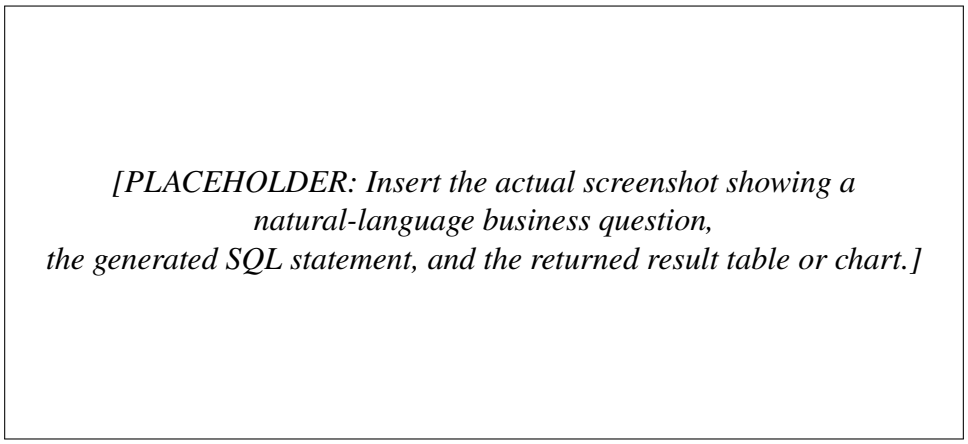


Figure 4.2: Natural-language-to-SQL interaction in the Streamlit interface (screenshot to be inserted)

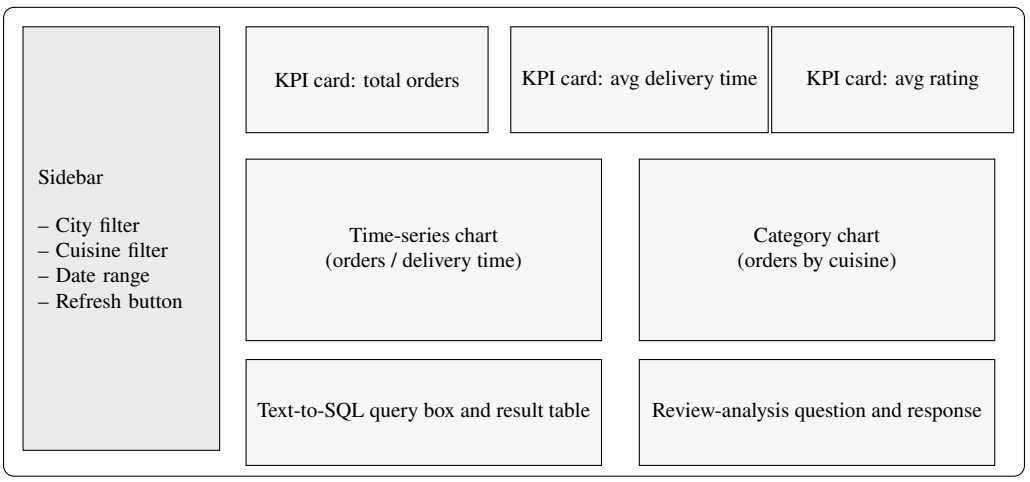


Figure 4.3: Dashboard layout of the Streamlit interface

4.8 Implementation Workflow

The implementation was carried out in the following sequence:

1. Prepare source files for customers, restaurants, menu items, orders, order items and reviews.
2. Upload the raw files to Amazon S3 and load them into Snowflake, retaining the raw representation in the Bronze layer.
3. Run dbt models to clean fields, standardise formats and build Silver tables.
4. Build Gold-level dimensions, facts and business marts for reporting.
5. Run dbt data-quality tests and check pipeline status.
6. Use Airflow to coordinate the sequence and schedule repeat runs.
7. Connect approved analytical tables to the Streamlit dashboard and the Text-to-SQL interface.
8. Send selected review text and the user's request to the Grok API and display the generated response.
9. Evaluate the system using the predefined functional and qualitative criteria.

CHAPTER 5

RESULTS AND DISCUSSION

CHAPTER 5

RESULTS AND DISCUSSION

This chapter reports what was observed during implementation and testing. Because measured results were not available when this draft was prepared, the tables below define the exact quantities to be recorded from actual runs; placeholders are marked and no unmeasured values are presented as final.

5.1 Experimental Setup

The evaluation uses the food-delivery dataset described in Chapter 4. Dataset size, row counts by table, Snowflake warehouse size, dbt and Airflow versions, execution environment and experimental duration are listed in Table 5.2. Testing follows a fixed procedure: (i) each functional task is executed end to end and its outcome recorded; (ii) dbt test results are collected; (iii) a fixed set of business questions is asked through Text-to-SQL and each generated query is compared with a manually verified expected answer; and (iv) a fixed set of review questions is asked and each answer is inspected for relevance and support by the supplied reviews.

5.2 Functional Results

Table 5.1 records whether the system can complete each major task.

5.3 Pipeline and Data-Quality Results

Table 5.2 records records loaded per table, dbt tests passed and failed, and any errors found during testing. If incremental processing is implemented, the handling of new or changed records is documented in the notes column.

5.4 AI Interaction Results

For the review feature, a fixed set of representative questions is used and both prompts and outputs are saved. A qualitative review records whether each answer is relevant

Table 5.1: Functional test results (outcomes to be recorded from actual runs)

Test ID	Feature tested	Expected result	Status
FT-01	Upload and load dataset into Snowflake Bronze	All six tables loaded with expected row counts	*[record]*
FT-02	Build Silver models via dbt	Cleaned tables created without test failures	*[record]*
FT-03	Build Gold dimensions, facts and marts	Marts queryable by the dashboard	*[record]*
FT-04	Execute dbt data-quality tests	All configured tests pass	*[record]*
FT-05	Run the Airflow DAG end to end	DAG completes with all tasks successful	*[record]*
FT-06	Answer review prompts through the Grok API	Relevant, grounded answers displayed	*[record]*
FT-07	Execute approved Text-to-SQL queries	Correct result returned for verified questions	*[record]*

Table 5.2: Dataset and pipeline summary (values to be recorded from actual runs)

Table / layer	Input rows	Output rows	Tests passed	Tests failed	Notes
Bronze (raw)	*[record]*	*[record]*	—	—	Raw copy retained
Silver (cleaned)	*[record]*	*[record]*	*[record]*	*[record]*	Type and null fixes
Gold: `fact_orders`	*[record]*	*[record]*	*[record]*	*[record]*	Star schema
Gold: dimensions	*[record]*	*[record]*	*[record]*	*[record]*	Conformed keys
Reviews (staging)	*[record]*	*[record]*	*[record]*	*[record]*	Text cleaned

to the supplied text, includes unsupported claims, or omits important evidence. These observations are not accuracy scores and are not generalised to all reviews.

For Text-to-SQL, business questions whose correct answers can be independently checked are used. Table 5.3 logs whether the generated SQL executes, uses approved tables and returns the expected result. Latency or API cost, if reported, is measured consistently with the stated test conditions.

Table 5.3: AI feature test log (outcomes to be recorded from actual runs)

ID	Feature	Question / input	Expected behaviour	Status
AI-01	Review analysis	"What are the most recurring delivery complaints?"	Themes supported by supplied reviews	*[record]*
AI-02	Review analysis	"Summarise positive feedback about food quality."	Grounded summary, no invented claims	*[record]*
AI-03	Text-to-SQL	"Which city has the highest average order value?"	Correct city with executed query	*[record]*
AI-04	Text-to-SQL	"List the top five restaurants by average rating (min. 50 orders)."	Correct ranking with join on approved marts	*[record]*

5.5 Discussion

The results must distinguish successful execution from answer quality. A pipeline can run successfully while still containing data-quality problems, and a query can execute without answering the user's question correctly. Likewise, a fluent review summary is not proof of sentiment-classification accuracy, nor is a sample of reviews statistically representative of all customers unless the sampling procedure supports that interpretation.

Observed failures are analysed together with their likely causes: for example, incorrect joins in generated SQL usually indicate that the schema description given to the model was incomplete, while unsupported claims in review answers usually indicate that the supplied context was too large or too loosely sampled. Changes made in response to such findings are recorded in Appendix A. Where lower latency is reported, it is interpreted together with answer quality rather than in isolation.

5.6 Limitations

The following limitations apply to the current version:

- The evaluation dataset is of restricted size and covers a single domain and season of data; results may not generalise.
- The review module uses direct prompting and is bounded by context length; it does not scale to very large review collections.
- Review output is not a validated sentiment classifier and carries no measured accuracy claim.
- The keyword-based SQL guard is a preliminary safeguard; unsafe patterns could in principle slip through or harmless queries be rejected.
- The system is batch-oriented; no streaming ingestion or production availability is claimed.
- External API usage incurs cost and rate limits, and only data permitted for external processing may be sent.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project demonstrated a structured approach for developing a cloud-based food-delivery analytics prototype that combines data engineering with AI-assisted access to business information and customer feedback. Amazon S3 is used for raw-file storage, Snowflake for layered analytical storage and processing, dbt for declarative transformations and data-quality checks, Airflow for workflow orchestration, and Streamlit for the user interface.

The work addresses the objectives stated in Chapter 1. The pipeline ingests six domain tables and organises them into Bronze, Silver and Gold layers; dbt tests provide automated quality checks; Airflow coordinates repeatable runs; the Grok-based feature accepts review text and user instructions to produce summaries and observations; and the Text-to-SQL feature translates business questions into guarded, read-only queries over curated analytical tables. Keeping the review path and the query path separate makes their evidence sources and limitations explicit. The value of the project is the integration of these components into one workflow, together with an evaluation methodology that records functional outcomes, test results and qualitative observations honestly. The strength of the final system should be judged from the documented results once the placeholders in Chapter 5 are replaced with measured values.

6.2 Future Scope

The current implementation is a batch-oriented prototype. The following extensions are proposed, to be introduced based on demonstrated needs and tested individually:

- Stronger SQL validation: replace keyword checks with SQL parsing, allow-list validation and strict database permissions.
- Review retrieval for larger datasets: add an embedding-based retrieval layer and evaluate it separately before describing the feature as retrieval-augmented.

- Human-reviewed evaluation sets for review answers and SQL questions, enabling quantitative evaluation of relevance and correctness.
- Operational monitoring of pipeline failures, API errors, query latency, token usage and estimated cost.
- Incremental refresh and scheduling so that only new or changed records are processed.
- Near-real-time ingestion and streaming updates for time-sensitive metrics.
- Broader analytics, such as trends by city, cuisine, delivery-time band and review topic, and studies of whether delivery-time shifts associate with review sentiment.
- Deployment hardening: access control, secrets management, logging, privacy checks and governance suitable for a larger user base.
- Generalisation of the architecture to adjacent domains such as e-commerce and hospitality.

Bibliography

[1] J. Schneider, C. Gröger, A. Lutsch, H. Schwarz, and B. Mitschang, "The lakehouse: State of the art on concepts and technologies," *SN Comput. Sci.*, vol. 5, no. 4, Art. no. 449, 2024, doi: 10.1007/s42979-024-02737-0.

[2] A. Mbata, Y. Sripada, and M. Zhong, "A survey of pipeline tools for data engineering," *arXiv:2406.08335*, 2024.

[3] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017, *arXiv:1706.03762*.

[4] T. B. Brown *et al.*, "Language models are few-shot learners," in *Proc. NeurIPS*, 2020.

[5] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020, *arXiv:2005.11401*.

[6] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. EMNLP*, 2018, pp. 3911–3921, doi: 10.18653/v1/D18-1425.

[7] M. Armbrust *et al.*, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," in *Proc. CIDR*, 2021.

[8] M. Armbrust *et al.*, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 1687–1692, 2020.

[9] N. Polyzotis, S. Karampatzakis, A. Kumar, and A. Tsoumakas, "Data management challenges in production machine learning," in *Proc. ACM SIGMOD*, 2017, pp. 1721–1734.

[10] S. Gupta, R. Ranjan, and S. N. Singh, "A comprehensive survey of retrieval-augmented generation (RAG): Evolution, current landscape and future directions," *arXiv:2410.12837*, 2024, doi: 10.48550/arXiv.2410.12837.

[11] A. Branco, D. Parada, M. Silva, F. Mendonça, S. S. Mostafa, and F. Morgado-Dias, "Sentiment analysis in Portuguese restaurant reviews: Application of transformer models in edge computing," *Electronics*, vol. 13, no. 3, Art. no. 589, 2024, doi: 10.3390/electronics13030589.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[13] L. Zhao *et al.*, "A survey for large language models," *arXiv:2303.18223*, 2023.

[14] J. Li *et al.*, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Proc. NeurIPS*, 2023.

[15] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual harnessing for efficient SQL synthesis," *arXiv:2405.16755*, 2024.

[16] V. Zhong, C. Xiong, and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," in *Proc. ACL*, 2017.

[17] Y. Chang *et al.*, "TAPEX: Table pre-training via learning a neural SQL solver," in *Proc. ACL*, 2022, pp. 53–65.

[18] Apache Software Foundation, "Apache Airflow documentation: What is Airflow?" [Online]. Available: https://airflow.apache.org/docs/. Accessed: Sep. 18, 2026.

[19] dbt Labs, "dbt documentation." [Online]. Available: https://docs.getdbt.com/. Accessed: Sep. 18, 2026.

[20] Amazon Web Services, "Amazon S3 documentation." [Online]. Available: https://docs.aws.amazon.com/s3/. Accessed: Sep. 18, 2026.

[21] Snowflake Inc., "Snowflake documentation." [Online]. Available: https://docs.snowflake.com/. Accessed: Sep. 18, 2026.

[22] Streamlit, "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/. Accessed: Sep. 18, 2026.

[23] xAI, "xAI API documentation (Grok)." [Online]. Available: https://docs.x.ai/. Accessed: Sep. 18, 2026.

[24] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

APPENDIX A: IMPLEMENTATION CONFIGURATIONS AND PROMPT TEMPLATES

6.3 Environment Variables

Credentials are supplied through environment variables and are never hard-coded in source files:

- Snowflake: `SNOWFLAKE_ACCOUNT`, `SNOWFLAKE_USER`, `SNOWFLAKE_PASSWORD`, `SNOWFLAKE_ROLE` (read-only role), `SNOWFLAKE_WAREHOUSE`, `SNOWFLAKE_DATABASE`, `SNOWFLAKE_SCHEMA`
- AWS S3: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `S3_BUCKET`
- Grok API: `XAI_API_KEY`

6.4 dbt Data-Quality Tests Configured

- `unique` on primary keys of all dimension and fact tables.
- `not_null` on key, timestamp and monetary columns of `fact_orders`.
- `relationships` from fact foreign keys to dimension keys.
- `accepted_values` on categorical columns such as order status and cuisine.

6.5 Review-Analysis Prompt Template

> *You are an analytics assistant. Use ONLY the review text supplied below. Do not invent facts, ratings, or customer names that are not present in the text. If the reviews are insufficient, say so. Question: {user_question} — Reviews: {review_samples}*

6.6 SQL Guard Rules (Preliminary)

- The statement must begin with `SELECT` (after trimming whitespace and comments).

- Forbidden keywords: `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `CREATE`, `TRUNCATE`, `GRANT`, `CALL`, `EXECUTE`.
- Execution uses a read-only role limited to approved Gold views, with a statement timeout and a row limit.

6.7 Recorded Versions

Item	Value
Python / Pandas	*[record]*
dbt Core	*[record]*
Apache Airflow	*[record]*
Snowflake account edition / warehouse size	*[record]*
Grok model identifier	*[record]*
Dataset source, licence, row counts, date range	*[record]*

ANNEXURE A: PUBLICATIONS AND INTELLECTUAL PROPERTY

This annexure records documentary evidence related to project outcomes, such as published or submitted papers, conference publications, patent applications, design registrations and other relevant intellectual property. The status of each item is clearly stated.

Table 6.1: Annexure A: Publications and intellectual property

Type	Title / description	Status
Research paper	"An Integrated Cloud Data Engineering and AI Analytics Platform for Food-Delivery Data" (A. Saw, A. Kolhe, A. Paradkar)	Draft prepared; journal / conference submission planned
Patent / design	—	None filed
Copyright / dataset	Project source code repository	To be registered on publication of the repository

ANNEXURE B: STUDENT CONTRIBUTION STATEMENT

For group projects, each student specifies a specific and verifiable contribution. The split below reflects the division of work carried out for this project and may be revised by the team to match the final division of labour.

Table 6.2: Annexure B: Student contribution statement

Student name	**Roll number**	**Contribution**
Amit Saw	*[roll number]*	Literature review, dataset collection, S3 and Bronze-layer ingestion setup
Ankit Kolhe	*[roll number]*	Snowflake modelling, dbt transformations and tests, Airflow DAG design
Aryan Paradkar	*[roll number]*	Streamlit interface, Grok review module, Text-to-SQL module, testing and documentation

ANNEXURE C: PROJECT OUTPUTS

Major outputs generated from the project are listed below:

- **Working prototype:** cloud pipeline (S3, Snowflake, dbt, Airflow) with Streamlit interface — *implemented; screenshots to be attached.*
- **Software / application:** Streamlit analytics application with dashboard, review analysis and Text-to-SQL modules — *implemented.*
- **Dataset:** curated food-delivery dataset in Bronze, Silver and Gold layers — *loaded; row counts to be recorded.*
- **Research publication:** draft research paper (Annexure A) — *in preparation.*
- **GitHub / project repository:** source code, dbt models and Airflow DAG files — *to be uploaded.*
- **Technology demonstration:** live demonstration of the dashboard, review analysis and natural-language querying — *scheduled with the project guide.*